PROJECT REPORT

Used Vehicle Resale Price Prediction

A Regression-Based Machine Learning Approach with Depreciation Curve Analysis

Dataset	CAR DETAILS FROM CARDEKHO (Kaggle)
Problem Type	Supervised Regression
Models Evaluated	12 regression algorithms
Deployment Platform	Streamlit Web Application
Report Date	July 2026

Table of Contents

1. Abstract

This project builds a machine learning system that predicts used car resale prices from CarDekho listing data. We trained and compared 12 different regression models — including linear regression, ridge/lasso, polynomial regression, SVM, decision trees, random forest, gradient boosting (XGBoost, LightGBM), KNN, and a neural network — all using the same data preprocessing steps so the comparison is fair. Each model was scored using R^2, Adjusted R^2, MAE, RMSE, and 5-fold cross-validation.

Besides just predicting a price, the app also shows how a car's value drops over time (depreciation) as it ages or racks up mileage, and lets you compare how well different brands hold their value. The whole thing is deployed as an interactive Streamlit web app.

2. Introduction

Figuring out what a used car is actually worth matters a lot — for buyers, sellers, and dealerships. It's a tricky problem because cars don't lose value in a straight line, some brands hold their value better than others, and there's a mix of feature types to deal with (categories like fuel type, transmission, seller type, ownership history, plus numbers like age and mileage). Most existing tools just spit out one price estimate and stop there.

This project goes a step further: instead of only predicting today's price, it also models how that price is expected to change as the car gets older or racks up more kilometers.

The project follows a standard, repeatable ML workflow: explore and clean the data, engineer useful features, train and compare 12 different regression models, pick the best one based on cross-validation results, and deploy it behind a simple web interface.

3. bjectives

1. To predict the resale price of a used vehicle from attributes including brand, fuel type, transmission, seller type, ownership history, vehicle age, and distance travelled.
2. To train and objectively compare twelve regression algorithms under a common preprocessing pipeline.
3. To evaluate each model using multiple complementary metrics — R^2, Adjusted R^2, MAE, RMSE, and five-fold cross-validation — rather than a single train/test split alone.
4. To explicitly model depreciation behaviour rather than restricting the output to a single price estimate.
5. To deploy the selected model as an interactive, user-facing web application.

4. Dataset Description

Source: "CAR DETAILS FROM CARDEKHO" dataset, publicly available on Kaggle, originally compiled from CarDekho used-vehicle listings.

Following data cleaning, the working dataset comprises approximately 3,500 records. Each record corresponds to a single vehicle listing and is described by the following attributes:

Feature	Type	Description
brand	Categorical	Vehicle manufacturer (18 brands; low-frequency brands grouped as "Other")
fuel	Categorical	Petrol, Diesel, CNG, LPG, or Electric
seller_type	Categorical	Individual, Dealer, or Trustmark Dealer
transmission	Categorical	Manual or Automatic
owner	Categorical (ordinal)	First Owner through Fourth & Above Owner, or Test Drive Car
age	Numerical (engineered)	Current year minus year of purchase
km_driven	Numerical	Total distance travelled, in kilometres
selling_price	Numerical (target)	Resale price in Indian Rupees (₹); log1p-transformed prior to training

Table 1. Description of dataset features.

Target transformation. As selling_price is right-skewed, a log1p transformation is applied prior to model training, with the inverse expm1 transformation applied to model outputs before evaluation and prediction. Consequently, all metrics reported in this document are expressed in real currency terms rather than in log space.

5. Methodology

5.1 Data Cleaning and Feature Engineering

- Duplicate listings and records with missing critical fields were removed.
- An age feature (age = current_year − year_of_purchase) was engineered in place of the raw purchase year, as it correlates more directly with depreciation.
- Low-frequency brands were consolidated into a single "Other" category to avoid sparse, high-dimensional encoding.
- A log1p transformation was applied to the right-skewed selling_price target variable.

5.2 Preprocessing Pipeline

Categorical features (brand, fuel, seller_type, transmission, owner) are encoded, and numerical features (age, km_driven) are scaled, within a single scikit-learn Pipeline object per model. Encapsulating preprocessing and estimation in one pipeline ensures that identical transformations are applied at both training and inference time; the deployed application accordingly calls model.predict() directly on a raw-feature input record, without risk of train–inference inconsistency.

5.3 Train–Test Split and Validation Strategy

- An 80:20 train–test split was used to reserve an independent evaluation set.
- Five-fold cross-validation was performed on the training set for every model, with the mean and standard deviation of R^2 reported, in order to assess the stability of each model's performance beyond a single data split.

6. Models Trained and Compared

Twelve regression models, spanning five algorithmic families, were trained under an identical preprocessing pipeline so that observed differences in performance reflect the modelling approach rather than inconsistencies in feature handling:

Family	Models
Linear / Regularised	Linear Regression, Ridge Regression, Lasso Regression
Polynomial	Polynomial Regression (degree 2)
Tree-Based	Decision Tree Regressor
Ensemble — Bagging	Random Forest Regressor
Ensemble — Boosting	Gradient Boosting, XGBoost, LightGBM
Kernel-Based	Support Vector Regressor (RBF kernel)
Instance-Based	K-Nearest Neighbors Regressor
Neural Network	Multi-Layer Perceptron (MLP) Regressor

Table 2. Regression models evaluated, grouped by algorithmic family.

7. Evaluation Metrics

- R^2 (Coefficient of Determination) — the proportion of variance in selling price explained by the model.
- Adjusted R^2 — R^2 adjusted for the number of predictors, controlling for artificial inflation as features are added.

- Mean Absolute Error (MAE) — the average absolute deviation, in ₹, between predicted and actual selling price.
- Root Mean Squared Error (RMSE) — an error measure, in ₹, that penalises larger deviations more heavily than MAE.
- Five-Fold Cross-Validated R^2 (mean ± standard deviation) — an estimate of how consistently each model generalises across different subsets of the training data.

8. Results

Table 3 presents the performance of all twelve models on the held-out test set, ordered by R^2 in descending order. The complete results, including MAE and cross-validation standard deviation, are provided in reports/model_comparison.csv.

Model	R^2	Adjusted R^2	RMSE (₹)	CV R^2 (mean)
Polynomial Regression (degree 2)	**0.787**	0.784	174,449	0.772
Support Vector Regressor (RBF)	0.778	0.776	177,830	0.767
Neural Network (MLP Regressor)	0.778	0.776	177,970	0.762
XGBoost Regressor	0.751	0.748	188,582	0.784
Ridge Regression	0.747	0.745	189,799	0.767
Linear Regression	0.747	0.744	190,034	0.767
Lasso Regression	0.740	0.737	192,642	0.765
LightGBM Regressor	0.738	0.735	193,306	0.764
Gradient Boosting Regressor	0.737	0.734	193,672	0.779
K-Nearest Neighbors Regressor	0.718	0.715	200,635	0.740
Random Forest Regressor	0.705	0.702	205,243	0.761
Decision Tree Regressor	0.547	0.543	254,048	0.693

Table 3. Comparative performance of all evaluated regression models on the held-out test set.

8.1 Key Observations

1. Polynomial Regression (degree 2) attains the highest single-split R^2 (0.787), indicating that the relationship between price and the available features contains mild non-linear structure that a second-degree expansion captures effectively.
2. The Support Vector Regressor (RBF kernel) and the Neural Network (MLP) follow closely, each achieving $R^2 \approx 0.778$, indicating that kernel-based and non-linear function-approximation methods perform comparably on this dataset.

3. XGBoost achieves the highest cross-validated R^2 (0.784) despite a comparatively lower single-split R^2 (0.751), indicating that it generalises most consistently across folds — a property that is arguably more informative than performance on any single split.
4. Random Forest and Decision Tree underperform relative to boosting methods and even relative to linear models. This is consistent with the modest size of the dataset (approximately 3,500 records) and the limited number of numerical features, a regime in which deep tree ensembles are prone to overfitting rather than to added predictive value.
5. The Decision Tree Regressor is the weakest-performing model overall (R^2 = 0.547), consistent with the known tendency of a single decision tree to overfit training data in the absence of ensembling.

8.2 Interpretation

Simpler models (and SVR) matched or beat the fancier tree-ensemble models on this dataset. XGBoost was the most consistent across different data splits though. Bottom line: bigger/more complex models don't automatically win — dataset size and feature quality matter too.

The app doesn't hardcode a model — it checks model_comparison.csv and best_model_name.json and automatically uses whichever one actually performed best.

9. Depreciation Curve Analysis

In addition to producing a single price estimate, the system evaluates the trained model across a range of input values, holding all other attributes constant, in order to characterise depreciation directly:

- Price versus Age — predicted price is computed for vehicle ages ranging from 0 to 15 years, with brand, fuel type, transmission, and distance travelled held fixed.
- Price versus Distance Travelled — predicted price is computed across a range of 0 to 220,000 kilometres, with age and all other attributes held fixed.
- Brand Value-Retention Comparison — predicted price trajectories for multiple brands are normalised to their respective values at age zero, expressed as a percentage, enabling a direct comparison of relative value retention across brands.

This approach extends the model beyond a single-value predictor into an exploratory analytical tool, enabling questions such as the expected loss in value over a five-year period, or the relative value retention of one brand compared to another, to be answered directly from the trained model's own predictions.

10. Application Architecture

The trained pipeline is deployed as a Streamlit web application (app.py), organised into three functional sections:

Section	Functionality
Price Prediction	Accepts vehicle attributes (brand, fuel, transmission, seller type, owner, year, distance travelled) via sidebar controls, and returns an instantaneous price estimate together with the name of the best-performing model, as identified during evaluation.
Depreciation Curves	Evaluates the model across a range of ages and distances travelled to render depreciation curves, together with a multi-brand value-retention comparison.
Model Comparison	Displays the complete twelve-model performance table, with the highest R^2/Adjusted R^2 and lowest MAE/RMSE values highlighted, alongside a sorted bar chart of R^2 by model.

Table 4. Functional sections of the deployed Streamlit application.

Model artefacts are cached using @st.cache_resource, and the comparison dataset is cached using @st.cache_data, to avoid unnecessary reloading from disk on repeated user interaction. The prediction routine constructs a single-row input record matching the training schema and invokes model.predict() directly; because preprocessing is embedded within the pipeline object, no discrepancy can arise between the transformations applied during training and those applied at inference time.

11. Project Structure

Path	Purpose
data/	Raw and cleaned/feature-engineered data files
src/eda_preprocessing.py	Exploratory data analysis, cleaning, and feature engineering
src/train_models.py	Training and comparative evaluation of all twelve regression models
src/depreciation_analysis.py	Generation of depreciation-curve plots
models/	Serialised model artefacts: best_model.pkl, all_models.pkl, best_model_name.json
reports/	Exploratory data analysis plots, model_comparison.csv/.json, and comparison and depreciation charts
app.py	Streamlit deployment application

Table 5. Repository structure and purpose of each component.

12. Technology Stack

Layer	Tools and Libraries
Data Processing / Machine Learning	pandas, numpy, scikit-learn, XGBoost, LightGBM
Visualisation	matplotlib, seaborn, Streamlit native charting components
Deployment	Streamlit (Streamlit Community Cloud or Hugging Face Spaces)

Table 6. Technology stack employed in the project.

13. Deployment

The application may be deployed without cost using Streamlit Community Cloud, by publishing the repository to a public GitHub account and configuring share.streamlit.io to point to app.py, or alternatively via Hugging Face Spaces using the Streamlit SDK. The trained artefacts (models/*.pkl and reports/model_comparison.csv) must be committed to the repository, as the application loads these files at runtime rather than retraining the models on startup.

14. Conclusion

This project shows a full, repeatable ML workflow: comparing multiple models with proper cross-validation, modeling depreciation instead of just a single price number, and shipping it as a working app people can actually use.

The results show that simpler models and kernel-based ones can match or beat tree ensembles on this dataset, while XGBoost was the most reliable across cross-validation. The bigger point: it's not true that "ensemble models always win" — the right model depends on your data, not just which algorithm sounds fanciest.

15. Future Work

- Incorporation of SHAP-based feature-importance explanations for individual predictions.
- Extension of the system to a second vehicle category, such as used motorcycles, to enable cross-category comparison.
- Addition of a currency and inflation adjustment mechanism.
- Redevelopment of the front-end using Flask or FastAPI in combination with React, to allow finer control over the user interface.

- Collection of a larger and more feature-rich dataset to determine whether tree-based ensemble methods outperform kernel- and linear-based models at greater scale.

16. References

1. **CarDekho Used Vehicle Dataset (Kaggle)** — the dataset this project uses, containing real used-car listings scraped from CarDekho.
2. **Scikit-learn paper (Pedregosa et al., 2011)** — the research paper behind scikit-learn, the main Python library used to build and evaluate most of the models here.
3. **XGBoost paper (Chen & Guestrin, 2016)** — the paper introducing XGBoost, one of the boosting models used in this project.
4. **LightGBM paper (Ke et al., 2017)** — the paper introducing LightGBM, another boosting model used here, known for being fast on large datasets.
5. **Streamlit Docs** — the official documentation for Streamlit, the tool used to turn the model into a web app.